

面向对象的模块化与信息隐藏

详细设计模块 · 第 2 讲

南京大学 · 软件工程课程组

模块	本讲主题
访问耦合	消息链问题 · 迪米特法则 · ISP
继承耦合	继承类型 · LSP · 组合优先
信息隐藏	类的职责 · 封装五类型
变更设计	为变更封装 · 权限最小化

今日学习路径

模块化（第 11 章）

1. 模块与耦合基础
2. 访问耦合的五个级别
3. Cascading Message 问题
4. **P4** 面向接口 · **P5** 迪米特法则 · **P6** ISP
5. 继承耦合的四种类型
6. **P7** 里氏替换原则 (LSP)
7. **P8** 组合优于继承

信息隐藏（第 12 章）

1. 信息隐藏的本质
2. 类的职责 = 状态 + 行为能力
3. 封装 A—E 五类型
4. ADT 抽象数据类型
5. Iterator 模式
6. **P9** 为变更而封装
7. **P10** 权限最小化 · **P11** OCP · **P12** DIP

本讲目标 — 理解面向对象设计中耦合与信息隐藏如何共同支撑**高内聚、低耦合**的模块化结构。

预习测试

以下哪种做法增加了类之间的耦合度？请选出所有正确项。

- A. 通过接口而非具体类引用对象
- B. 直接访问另一个对象的公开字段
- C. 子类覆盖父类方法并改变返回语义
- D. 使用 `a.getB().getC().doSomething()`
- E. 将集合字段声明为 `public`
- F. 在接口中只放入调用方需要的方法
- G. 子类在父类方法中增加新的前置条件

答案：B、C、D、E、G — 这些做法分别对应访问耦合过高、LSP 违反、Cascading Message、封装缺失以及契约强化。

Part 1

模块化基础与耦合概念

Module · Coupling · Cohesion

什么是模块（Module）

A **module** is a self-contained unit of software that can be independently designed, implemented, and tested.

在面向对象中，模块有三种粒度：

粒度	对应构件	职责边界
方法级	Method / Function	单一操作
类级	Class	状态 + 行为
包级	Package / Namespace	相关类集合

模块化的两个核心指标

耦合（Coupling）

模块之间的依赖强度
→ 越低越好

内聚（Cohesion）

模块内部元素的关联程度
→ 越高越好

设计目标：高内聚、低耦合

面向对象的耦合类型

在 OO 系统中，耦合来自两个方向：

访问耦合

Component Coupling

类 A **使用**类 B 的服务时产生

- 调用方法
- 访问字段
- 作为参数传递
- 通过消息链跨越多跳

继承耦合

Inheritance Coupling

类 A **继承**类 B 时产生

- 子类依赖父类的实现细节
- 父类修改可能破坏子类
- 比访问耦合更深、更隐蔽

继承是"白盒复用"——子类可以看到父类的内部实现；访问耦合是"黑盒复用"——只依赖接口。

Part 2

| 访问耦合

五个级别 · Cascading Message · P5 迪米特法则 · P4 面向接口 · P6 ISP

访问耦合的五个级别

耦合强度从高到低：

级别	名称	说明	示例
5（最高）	隐式访问耦合	依赖全局变量或 Singleton	<code>GlobalConfig.instance.value</code>
4	实现中访问耦合	访问另一类的私有实现细节	反射访问 private 字段
3	成员变量访问耦合	直接读写另一类的 public 字段	<code>order.price = 100</code>
2	参数变量访问耦合	通过方法参数传递，依赖具体类型	<code>void process(ConcreteOrder o)</code>
1（最低）	无访问耦合	无直接依赖	—

级别 3—5 打破封装，应重构为通过方法（消息）访问。级别 1—2 是可接受范围。

Cascading Message（消息链）问题

```
// 问题代码：消息链 (Train Wreck)
String city = customer
    .getAddress()
    .getCity()
    .toUpperCase();
```

问题：Customer 的调用方知道了：

- Customer 有 Address
- Address 有 City
- City 是 String

三层实现细节全部暴露！

两种修复方案

方案 A：引入局部变量

```
Address addr = customer.getAddress();
City city = addr.getCity();
String name = city.toUpperCase();
```

可读性提升，但耦合依然存在。

方案 B：委托（推荐）

```
// 在 Customer 中封装
String city = customer.getCityName();
```

调用方只需知道 Customer。

P5 · 迪米特法则 (Law of Demeter)

只与你的直接朋友交谈。

A method should only call methods on: itself, its parameters, objects it creates, and its direct fields.

合法的调用对象

1. `this` (自身)
2. 方法的参数对象
3. 方法内新创建的对象
4. 类的成员变量 (直接持有)

违反示例

```
// 违反: 通过 getB() 拿到中间对象再调用  
a.getB().doSomething();
```

```
// 合规: 委托给 B 的操作封装在 A 中  
a.delegateToB();
```

迪米特法则是对访问耦合级别的直接约束: 避免消息链, 减少对间接对象的依赖。

P4 · 面向接口编程

Program to an interface, not an implementation.

```
// 不好：依赖具体类
ArrayList<String> list = new ArrayList<>();

// 好：依赖接口
List<String> list = new ArrayList<>();
```

好处：

- 调用方只依赖**接口契约**，不依赖实现
- 切换实现类无需修改调用方
- 支持多态、Mock 测试

面向接口编程将访问耦合从**参数变量级**降低到仅依赖接口，是降低耦合的核心手段。

P6 · 接口隔离原则 (ISP)

Clients should not be forced to depend on interfaces they do not use.

肥接口问题

```
interface Worker {  
    void work();  
    void eat();    // Robot 不需要!  
    void sleep(); // Robot 不需要!  
}  
  
class Robot implements Worker {  
    public void work() { ... }  
    public void eat() { throw new UnsupportedOperationException(); }  
    public void sleep(){ throw new UnsupportedOperationException(); }  
}
```

ISP 重构

```
interface Workable {  
    void work();  
}  
  
interface Feedable {  
    void eat();  
    void sleep();  
}  
  
class Robot implements Workable { ... }  
class Human implements Workable,  
                        Feedable { ... }
```

接口细化后，**Robot** 不再被迫实现无关方法，调用方只依赖自己真正需要的接口。

Part 3

| 继承耦合

继承类型 · P7 LSP · P8 组合优先

继承耦合的四种类型

类型	说明	耦合强度	推荐度
Modification（修改）	子类覆盖父类方法并改变行为语义	最高	✗ 避免
Refinement（精化）	子类调用 <code>super</code> 并在前后添加行为，不改变语义	中	△ 谨慎
Extension（扩展）	子类只新增方法，不覆盖父类任何方法	低	✓ 安全
Nil（无实现）	父类是纯抽象接口，子类提供实现	最低	✓✓ 最佳

从设计视角：**Nil → Extension → Refinement → Modification** 是耦合由低到高的排列。推荐优先使用抽象父类（Nil）和扩展继承（Extension）。

P7 · 里氏替换原则 (LSP)

子类型必须能够替换其父类型而不改变程序的正确性。

— Barbara Liskov, 1987

经典反例：正方形 ≠ 长方形

```
class Rectangle {  
    void setWidth(int w) { ... }  
    void setHeight(int h) { ... }  
    // 不变式: w 与 h 独立  
}  
  
class Square extends Rectangle {  
    void setWidth(int w) {  
        super.setWidth(w);  
        super.setHeight(w); // 破坏不变式!  
    }  
}
```

为什么违反 LSP?

```
void resize(Rectangle r) {  
    r.setWidth(5);  
    r.setHeight(3);  
    // 期望面积 = 15  
    assert r.area() == 15; // Square 中失败!  
}
```

Square 在几何上是 **Rectangle** 的子集，但在行为契约上不能替换。"IS-A" 关系应由行为定义，而非数学定义。

LSP 反例：企鹅与鸟

```
class Bird {  
    void fly() { ... } // 所有鸟都会飞?  
}  
  
class Penguin extends Bird {  
    @Override  
    void fly() {  
        throw new UnsupportedOperationException("企鹅不会飞!");  
    }  
}  
  
// 调用方:  
void makeBirdFly(Bird b) {  
    b.fly(); // Penguin 会抛异常!  
}
```

修复方案

```
interface Flyable {  
    void fly();  
}  
  
class Sparrow extends Bird  
    implements Flyable {  
    public void fly() { ... }  
}  
  
class Penguin extends Bird {  
    // 不实现 Flyable  
    void swim() { ... }  
}
```

用接口分离代替强制继承层次，Penguin 不再承诺 fly()，LSP 自然满足。

LSP 的契约含义

LSP 要求子类遵守父类的行为契约：

契约要素	LSP 要求
前置条件	子类前置条件 $\leq$ 父类（子类不能更严格）
后置条件	子类后置条件 $\geq$ 父类（子类保证更多）
不变式	子类必须维护父类的所有不变式

违反示例（前置条件更严格）

```
// 父类：接受任意正整数
void setAge(int age) { assert age > 0; }

// 子类：要求年龄 < 150（更严格）
void setAge(int age) {
    assert age > 0 && age < 150; // 违反!
}
```

记忆口诀

子类前置放宽，后置加强，
不变式维持——这才叫替换。

P8 · 组合优于继承

Favor object composition over class inheritance.

— GoF, **Design Patterns**, 1994

继承的问题

- **白盒复用**：子类依赖父类实现细节
- 父类修改可能**破坏**所有子类
- 继承层次**难以扩展**（多个维度变化时）
- 强迫遵守 IS-A 语义

组合的优势

- **黑盒复用**：只依赖接口
- 运行时可**动态切换**组合对象
- 职责分离更清晰
- 无需父类 IS-A 约束

```
// 继承: Stack extends Vector (历史错误)
// 组合: Stack 内部持有 Vector
class Stack<T> {
    private Vector<T> storage = new Vector<>();
    public void push(T item) { storage.add(item); }
    public T pop() { return storage.remove(...); }
}
```

Part 4

面向对象的信息隐藏

类的职责 · 封装 A—E · ADT

信息隐藏的本质

每个模块应该对其他模块隐藏一个重要的设计决策。

— David Parnas, 1972

两类秘密 (Secrets)

主要秘密 (Primary Secret)

= 职责变更的可能性

→ 这个类**应该做什么**会变化吗?

次要秘密 (Secondary Secret)

= 实现变更的可能性

→ 这个类**如何做**会变化吗?

为什么要隐藏?

变化是软件的常态。

将变化的可能性**封装在模块内部**，使得修改只影响该模块，而不波及其他模块。

信息隐藏 = **隔离变化** = 降低耦合

类的职责

类的职责 = 状态 (State) + 行为能力 (Behavioral Capability)

状态 (State)

- 类所持有的数据
- 描述对象在某一时刻的情况
- 通过字段 (field) 表示

```
class BankAccount {  
    private double balance; // 状态  
    private String owner;   // 状态  
}
```

行为能力 (Behavioral Capability)

- 类能对外提供的操作
- 改变状态或查询状态
- 通过方法 (method) 表示

```
class BankAccount {  
    void deposit(double amount) { ... }  
    void withdraw(double amount) { ... }  
    double getBalance() { ... }  
}
```

类的**接口** (interface) = 对外承诺的行为能力；类的**实现** (implementation) = 内部状态与算法细节。

封装 = 接口 + 实现

OO 接口的五个要素：

要素	说明	示例
方法名	操作的标识符	deposit
参数	输入类型与顺序	(double amount)
返回类型	输出类型	void
不变式	类的恒定约束	balance >= 0
异常	错误情况契约	throws InsufficientFundsException

实现的五类细节（应被隐藏）：

类型	说明
Data	用什么数据结构存储状态
Structure	数据的内部组织方式
Other objects	内部持有哪些对象引用
Type	字段/变量的具体类型
Change/Vary	可能变化的算法或策略

封装的五种类型（A—E）

类型	隐藏内容	核心手段
A — 数据类型（ADT）	内部数据表示	访问器/修改器替代 public 字段
B — 内部结构	数据的组织结构	不暴露内部容器/数组
C — 其他对象引用	内部持有哪些对象	不直接返回内部对象引用
D — 类型信息（LSP）	对象的具体运行时类型	面向接口，多态调度
E — 潜在变更	可能变化的实现策略	策略模式、依赖注入

类型 A—C 处理**当前实现细节**的封装；类型 D—E 处理**未来变化**的封装。这五种类型覆盖了面向对象封装的完整语义。

类型 A: ADT — 抽象数据类型

ADT 定义的是**概念**，不是实现。

公开字段（违反封装 A）

```
class Point {  
    public double x; // 暴露实现：直角坐标  
    public double y;  
}  
  
// 调用方直接依赖坐标系！  
point.x = 3.0;
```

若改为极坐标实现，**所有调用方都要修改**。

ADT 封装（推荐）

```
class Point {  
    private double x, y;  
  
    // 直角坐标接口  
    double getX() { return x; }  
    double getY() { return y; }  
  
    // 极坐标接口（同一 ADT）  
    double getR() {  
        return Math.sqrt(x*x + y*y);  
    }  
    double getTheta() {  
        return Math.atan2(y, x);  
    }  
}
```

访问器与修改器 (Accessor / Mutator)

用方法替代直接的 public 字段访问：

反模式：直接字段访问

```
class Person {  
    public String name;  
    public int age;  
}  
  
// 调用方：  
person.age = -5; // 无法验证!
```

问题：

- 无法添加验证逻辑
- 字段类型变更破坏调用方
- 无法懒加载或计算

正确做法：访问器 + 修改器

```
class Person {  
    private String name;  
    private int age;  
  
    public int getAge() { return age; }  
  
    public void setAge(int age) {  
        if (age < 0 || age > 150)  
            throw new IllegalArgumentException();  
        this.age = age;  
    }  
}
```

类型 B/C：隐藏内部结构与对象引用

问题：暴露内部集合（违反 B+C）

```
class Route {  
    public List<Stop> stops; // 直接暴露!  
}  
  
// 调用方可以任意修改内部状态：  
route.stops.add(newStop); // 绕过业务规则  
route.stops.clear();      // 危险!
```

修复：使用 Iterator 或受控接口

```
class Route {  
    private List<Stop> stops = new ArrayList<>();  
  
    // 只暴露迭代能力  
    public Iterator<Stop> getStops() {  
        return Collections  
            .unmodifiableList(stops)  
            .iterator();  
    }  
  
    // 受控的修改接口  
    public void addStop(Stop s) {  
        // 可以在这里加业务验证  
        stops.add(s);  
    }  
}
```

Iterator 模式是封装集合内部结构的标准手段——调用方只知道可以遍历，不知道内部是 `List`、`Set` 还是数组。

类型 D：隐藏类型信息（LSP）

调用方应该依赖接口，而不需要知道对象的具体类型。

违反封装 D：instanceof 判断

```
void process(Shape shape) {  
    if (shape instanceof Circle) {  
        ((Circle) shape).drawCircle();  
    } else if (shape instanceof Square) {  
        ((Square) shape).drawSquare();  
    }  
}
```

问题：每增加一种 `Shape`，就要修改这段代码。

正确：多态调度

```
interface Shape {  
    void draw(); // 统一接口  
}  
  
class Circle implements Shape {  
    public void draw() { /* 画圆 */ }  
}  
  
class Square implements Shape {  
    public void draw() { /* 画方 */ }  
}  
  
void process(Shape shape) {  
    shape.draw(); // 不知道也不需要知道类型  
}
```

类型 E：封装潜在变更

将可能变化的部分隔离为独立模块，其余部分依赖稳定的接口。

问题：变化硬编码在类内部

```
class Sorter {  
    void sort(int[] arr) {  
        // 直接写死冒泡排序  
        // 若要换快速排序，必须修改本类  
        bubbleSort(arr);  
    }  
}
```

策略模式封装变更

```
interface SortStrategy {  
    void sort(int[] arr);  
}  
  
class Sorter {  
    private SortStrategy strategy;  
  
    Sorter(SortStrategy s) {  
        this.strategy = s;  
    }  
  
    void sort(int[] arr) {  
        strategy.sort(arr); // 委托给策略  
    }  
}
```

封装类型 E 与**开闭原则（OCP）**直接对应：对扩展开放，对修改关闭。

Part 5

| 为变更而设计

P9 变更封装 · P10 权限最小化 · P11 OCP · P12 DIP · P13 SRP · P14 SOFA

P9 · 为变更而封装 (Encapsulate What Varies)

找到系统中变化的部分，将其封装起来，使变化不影响不变的部分。

识别变化点的三个问题

- 1. 哪些需求最可能改变？
(算法？ 数据格式？ 业务规则？)
- 2. 哪些部分跨越边界共享？
(多个类都依赖的实现细节)
- 3. 哪些部分已经变过多次？
(历史修改记录是最好的指示)

封装变化的手段

变化类型	封装手段
算法/策略	策略模式
对象创建	工厂方法/抽象工厂
对象结构	组合模式
跨对象行为	命令/访问者模式
数据格式	Adapter 模式

P10 · 权限最小化

Minimize the accessibility of classes and members.

— **Effective Java**, Joshua Bloch

Java 的四个访问级别

修饰符	可访问范围
private	仅本类内部
(package-private, 无修饰符)	同一包内
protected	同一包 + 子类
public	任何地方

原则：从最严格的 `private` 开始，只在确实需要时才放宽。

为什么要最小化权限？

每增加一个 `public` 成员，就等于增加了一个对外的承诺。这个承诺未来很难撤回，因为可能有调用方依赖它。

权限最小化的好处：

- 接口更小 → 耦合更低
- 内部实现可以自由修改
- 降低 API 的维护成本
- 提高安全性

P11 · 开闭原则 (OCP)

Software entities should be open for extension, but closed for modification.

— Bertrand Meyer

违反 OCP 的典型模式

```
// 每次增加新类型都要修改
class DiscountCalculator {
    double calculate(Order order) {
        if (order.type == VIPORDER)
            return order.total * 0.8;
        else if (order.type == NEWUSER)
            return order.total * 0.9;
        // 增加新类型需修改本类...
    }
}
```

符合 OCP 的重构

```
interface DiscountPolicy {
    double apply(double total);
}

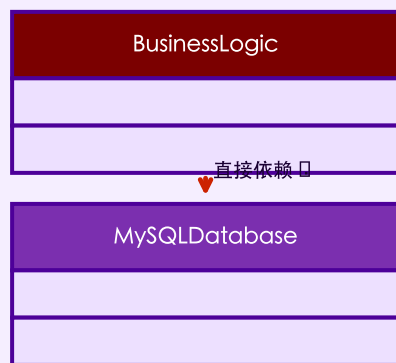
class VipDiscount implements DiscountPolicy {
    public double apply(double t) {
        return t * 0.8;
    }
}

class DiscountCalculator {
    private DiscountPolicy policy;
    // 新增类型只需新增实现类
    double calculate(Order o) {
        return policy.apply(o.getTotal());
    }
}
```

P12 · 依赖倒置原则 (DIP)

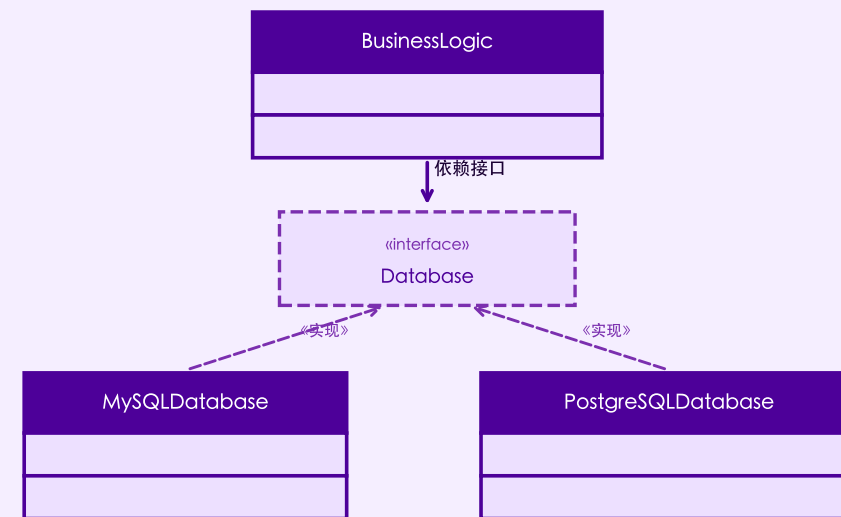
高层模块不应依赖低层模块；两者都应依赖抽象。
抽象不应依赖细节；细节应依赖抽象。

违反 DIP



```
class BusinessLogic {
    private MySQLDatabase db; // 依赖具体类!
    void process() {
        db.query(...);
    }
}
```

符合 DIP



```
class BusinessLogic {
    private Database db; // 依赖接口
    BusinessLogic(Database db) {
        this.db = db; // 依赖注入
    }
}
```

P13 · 单一职责原则 (SRP)

A class should have only one reason to change.

— Robert C. Martin

违反 SRP

```
class Employee {  
    String name;  
    double salary;  
    // 职责1: 薪资计算 (HR 规则)  
    double calculatePay() { ... }  
    // 职责2: 数据库持久化 (IT 基础设施)  
    void saveToDatabase() { ... }  
    // 职责3: 报表生成 (财务需求)  
    String generateReport() { ... }  
}
```

三个不同的"变化原因"混在一个类中。

SRP 重构

```
class Employee {           // 只管员工数据  
    String name;  
    double salary;  
}  
class PayCalculator {      // 只管薪资计算  
    double calculate(Employee e) { ... }  
}  
class EmployeeRepository { // 只管持久化  
    void save(Employee e) { ... }  
}  
class EmployeeReport {    // 只管报表  
    String generate(Employee e) { ... }  
}
```

SRP 与信息隐藏直接关联：每个类只隐藏一个"变化原因" (= Parnas 的"设计决策")。SRP 违反的症状是"God Class"——一个类知道太多、做太多。

P14 · SOFA：方法级设计原则

SOFA = Short · One thing · Few arguments · Abstraction level consistency

— 面向方法粒度的设计原则（与 SOLID 的类粒度原则互补）

S — Short（方法要短）

- 方法体通常 $\leq 20-30$ 行
- 过长 $\rightarrow$ 分解为命名清晰的辅助方法

O — One thing（只做一件事）

- 方法名能用一个短句完整描述其行为
- 若需要"and"才能描述 $\rightarrow$ 考虑拆分

F — Few arguments（参数要少）

- 参数越多，调用方知道的细节越多 $\rightarrow$ 耦合越高
- 超过 3 个参数时考虑引入参数对象

A — Abstraction level consistency（抽象层次一致）

```
// 违反 A：高层业务逻辑混入低层实现
void processOrder(Order o) {
    validateOrder(o);           // 高层
    db.execute("INSERT ...");   // 低层!
}

// 修复：把低层细节封装为方法
void processOrder(Order o) {
    validateOrder(o);
    persistOrder(o); // 统一在高层
}
```

原则汇总（P1–P7）：信息隐藏基础 · 访问耦合 · 继承耦合

完整编号体系：P1–P4 见 04–01（信息隐藏基础），P5–P14 见本讲（面向对象设计）

编号	缩写	原则	核心思想	SWEBOK
P1	—	全局变量有害	全局变量穿透所有模块，是隐藏的反义词	§1.4 Encapsulation
P2	—	优先显示表达	用接口/文档显式表达约束，不靠隐式约定	§1.4
P3	DRY	不要重复	同一决策不在多处编码；重复=多处修改	§1.4
P4	—	面向接口编程	依赖接口而非实现，降低访问耦合	§1.4 Sep. of I&I; §5.4
P5	LoD	迪米特法则	只与直接朋友交谈，避免消息链	§5.4 OOD
P6	ISP	接口隔离	接口细化，客户端只依赖需要的方法	§5.4 SOLID–I
P7	LSP	里氏替换	子类前置放宽、后置加强、不变式维持	§5.4 SOLID–L

原则汇总（P8—P14）：封装变更 · SOLID · SOFA

编号	缩写	原则	核心思想	SWEBOK
P8	—	组合优于继承	黑盒复用，避免继承耦合	§5.4 GoF
P9	—	为变更而封装	隔离可变部分，稳定不变部分	§1.4 Encapsulation
P10	—	权限最小化	从 <code>private</code> 开始，按需放宽	§1.4
P11	OCP	开闭原则	对扩展开放，对修改关闭	§5.4 SOLID-O
P12	DIP	依赖倒置	高层依赖抽象，不依赖具体实现	§5.4 SOLID-D
P13	SRP	单一职责	一个类只有一个变化原因	§5.4 SOLID-S
P14	SOFA	方法设计	Short · One thing · Few args · Abstraction	§5.4 SOFA

记忆结构：P1—P4 防止"隐式依赖"；P5—P8 控制"耦合来源"；P9—P13 管理"变化封装"；P14 约束"方法粒度"。

Part 6

| LLM 与面向对象设计

实证研究 · 能力边界 · 工具推荐

LLM 对耦合/内聚的理解：实证发现

"当前 LLM 是有效的辅助设计工具，但不是自主设计主体。"
— arXiv:2511.20933, **Hierarchical Evaluation of Software Design Capabilities of LLMs** (2024)

实验设置 (DeepSeek-R1 系列)

三种引导级别逐步增加噪声：

- 1. **Verification** — 验证给定答案
- 2. **Guided Generation** — 有提示的生成
- 3. **Open-ended Generation** — 完全开放

关键发现：耦合

- 有引导时：基准正确率尚可
- 开放场景 + 噪声：F1 降幅 >50%
- 模型对耦合问题存在"认知捷径" (Cognitive Shortcutting)

关键发现：内聚 vs 耦合

维度	LLM 表现	原因
内聚分析	较稳定	局部信息足够
耦合推理	噪声下崩溃	需要跨类全局推理

教学含义：让 LLM 帮你评估"这段代码的内聚性"通常有效；让它评估"跨多个类的耦合关系"需要额外的上下文和结构化提示，结果不可盲信。

📖 SWEBOK v4 §1.4 · Coupling | §1.4 · Cohesion | §6.3 · Quality Analysis

LLM 违反模块化的规律 ①：两大结构性缺陷

量质反比定律 (Volume–Quality Inverse Law)

"代码体积几乎是结构退化的完美预测因子。"

— arXiv:2605.02741, **AI–Generated Smells**, 2025

- LLM 生成代码比人类基线多 **42%–85%** 代码异味 (arXiv:2510.03029)
- 模型能力越强 → 代码体积越大 → **结构越臃肿、耦合越高**

推论：让 LLM 写更长、更完整的代码，往往意味着更多结构性问题，而非更高质量。

模块化悖论 (The Modular Imperative)

"即使明确提供模块化指导，LLM 仍倾向于把新功能加入已有类而非新建职责单一的类。"

— Harvard ICFP/SPLASH 2025

直接违反：

- **SRP** (单一职责)：一个类承担多个变化原因
- **信息隐藏**：把不同变化原因混入同一模块
- **P9 封装变更**：变化点未被隔离

根本原因：LLM 的训练目标是预测下一个 token，而非最小化模块间耦合——它"看到"的是局部代码，感知不到跨文件的全局模块化约束。

LLM 违反模块化的规律 ②：代码异味修复能力

异味类型	LLM 能力	研究来源
简单异味 (Magic Number、Empty Catch)	✅ GPT-4o 解决 ~62%	IEEE TSE 2025
Broken Modularization	❌ 显著劣于人类	ACM TOSEM 2024
Deficient Encapsulation	❌ 显著劣于人类	ACM TOSEM 2024
Multifaceted Abstraction	❌ 显著劣于人类	ACM TOSEM 2024
架构级异味 (God Class 等)	⚠️ iSMELL 工具辅助可达 F1=75%	ASE 2024
架构级异味 (SmellBench 2026)	⚠️ 最佳解决率 47.7%；63.1% 检测为误报	arXiv:2605.07001
少样本重构 (Few-shot)	⚠️ 异味减少率 42.97% (优化提示词)	arXiv:2511.21788

可用：表层语法级异味 (Magic Number、空 catch) 修复成功率可接受，适合作为 CI 流水线的自动修复步骤。

工具推荐：iSMELL = LLM + 7 专家静态分析工具组合，God Class / Feature Envy 检测 F1 比纯 LLM 提升 +35% (ASE 2024)。

SOLID 检测实证 ①：实验设计与关键数据

"Are We SOLID Yet?" — arXiv:2509.03093, September 2025

实验设置

- 任务：**自动检测 Java 代码中的 SOLID 原则违反
- 模型：**GPT-4o Mini、GPT-4o、Claude 3.5、Llama 3、Gemini 1.5 等
- Prompt 策略：**零样本 / 少样本 / 集成投票 / 基于示例

各原则最优 F1

SOLID 原则	最佳 Prompt 策略	最优 F1
OCP（开闭）	集成投票（Ensemble）	71.2%
DIP（依赖倒置）	示例提示（Example-hints）	64.8%
SRP（单一职责）	少样本	58.3%
ISP（接口隔离）	零样本	51.7%
LSP（里氏替换）	零样本	44.1%

核心结论

- GPT-4o Mini 总体优于更大模型**
结构化推理步骤比参数规模更关键——小模型 + 好 Prompt > 大模型 + 零样本
- 复杂度是核心瓶颈**
每增加 5 行代码，F1 下降约 15%
LSP 和 SRP 在复杂类体系中准确率接近随机
- OCP / DIP 是最适合 LLM 辅助检测的原则**，因为其违反模式局部、规律性强；SRP / LSP 需要跨上下文语义理解，LLM 明显力不从心。

SOLID 检测实证 ②：复杂度陷阱与设计模式识别

复杂度退化曲线

代码规模	检测表现
≤ 10 行简单方法	✅ OCP/DIP 可用 (F1 ≥ 65%)
每 +5 LOC	❌ F1 下降约 15%
复杂类体系	❌ LSP/SRP 准确率接近随机

实践推论：LLM 的 SOLID 检测能力随代码复杂度指数级退化。只在小粒度代码审查中使用，不可用于全库扫描。

设计模式检测 (2026)

- **LLM-Based Pattern Detection** (arXiv:2502.18458)：自动跨代码库识别设计模式实例，专注于**类角色识别**
- **Do LLMs Understand Design Patterns?** (arXiv:2501.04835)：Code LLM 在某些模式类型存在**系统性偏见**，识别 / 理解 / 生成三阶段均有弱点

实践指导

- ✅ 扫描**小型方法** (≤30 行) 的 OCP / DIP 违反
- ⚠️ 复杂类体系需配合静态分析工具 (如 iSMELL)
- ❌ 不可依赖 LLM 做全库 SRP / LSP 合规扫描

LLM 辅助 OO 设计 ①：有效用法

✓ 生成单类骨架（信息隐藏约束）

设计一个 `BankAccount` 类：

- 主要秘密（信息隐藏）：余额的存储方式
 - 约束：字段全部 `private`，只暴露访问器和业务方法，余额不能为负
- 请用 `Java` 写出类骨架和 `Javadoc`。

✓ 识别简单代码异味并重构

以下代码有哪些违反 `SRP` 或封装的地方？
请按 `A/B/C/D/E` 封装类型分类，
并给出重构方案。[粘贴代码]

✓ 检测小型方法的 `SOLID` 违反（`OCP/DIP` 优先）

以下方法是否违反开闭原则（`OCP`）？
请说明违反的具体位置并给出重构方案。
[粘贴代码，≤30 行]

✓ 生成 `PlantUML` / `SVG` 类图初稿

适用于单类或少量相关类；多类关系需人工校验。

✓ 多智能体 `UML` 生成（`NOMAD` 方式）

将类图生成拆解为目标导向子任务，交给不同角色智能体协作，优于单次对话生成。

✓ 封装五类型快速诊断

请检查以下类，按封装类型 `A`（`ADT`）、`B`（结构）、`C`（对象引用）、`D`（类型）、`E`（变更）分类列出所有违反封装的地方。
[粘贴代码]

共同要点：有效 `Prompt` 必须包含设计约束（`SOLID` 原则编号、封装类型、变化点），而不只是描述需求。

LLM 辅助 OO 设计 ②：危险用法与边界

需人工验证 (⚠️)

多类关系设计

LLM 生成的多类图经常出现：多重度错误、关联方向反转、组合/聚合混淆——每张图都需逐条审查。

耦合评估

复杂场景下跨类耦合推理不可靠 (F1 降幅 >50%)，需人工核查访问耦合级别 (Level 1–5)。

架构级代码异味检测

SmellBench (2026)：63.1% 的检测结果是**误报**，必须结合静态分析工具 (iSMELL 等) 交叉验证。

不可依赖 (❌)

职责边界划分 (无上下文)

LLM 不知道"什么会变"，无法自主做出信息隐藏决策——必须由人类先识别变化点，再让 LLM 落地。

库级全系统模块化设计

ProgramBench (2026)：全任务完成率仅 **3%**；LLM 无法感知跨文件的全局模块化约束。

最佳实践：把 LLM 当"知识渊博但不了解你业务的**初级设计师**"——它熟悉 SOLID / GoF / 封装，但需要你提供**领域约束**和**变化点**，才能生成有价值的初稿。人类保留**职责边界划分**、**架构级重构**、**设计理由**的最终决策权。

课后作业

编程练习

给定如下代码，识别所有**封装问题**并重构：

```
class Library {
    public List<Book> books = new ArrayList<>();
    public String name;

    public Book findBook(String isbn) {
        for (Book b : books) {
            if (b.isbn.equals(isbn))
                return b;
        }
        return null;
    }
}

class Book {
    public String isbn;
    public String title;
    public double price;
}
```

分析要求

1. 列出所有违反封装的地方（对应 A—E 哪种类型？）
2. 该设计违反了哪些原则（LSP/ISP/权限最小化/迪米特法则）？
3. 重构后的代码需满足：
 - `books` 集合不可从外部直接修改
 - `isbn`、`title`、`price` 通过访问器访问
 - `findBook` 在无结果时返回 `Optional<Book>`

提交 GitHub 链接，含重构前后对比与原则对照说明。

小结

面向对象的模块化与信息隐藏

模块化

- 耦合 = 模块间依赖强度（越低越好）
- 访问耦合 5 级别：隐式 > 实现 > 成员 > 参数 > 无
- 消息链破坏 **P5** 迪米特法则 → 委托
- **P6** ISP：接口不强迫客户端依赖不需要的方法
- 继承耦合：Modification > Refinement > Extension > Nil
- **P7** LSP：子类前置放宽、后置加强、不变式维持
- **P8** 组合 = 黑盒复用，更灵活

信息隐藏

- 每个模块隐藏一个**设计决策**（**P13** SRP：只有一个变化原因）
- 类职责 = 状态 + 行为能力
- 封装 A（ADT）B（结构）C（对象）D（类型）E（**P9** 变更）
- Iterator 隐藏集合结构
- **P10** 权限最小化：从 private 开始
- **P11** OCP / **P12** DIP 是封装变更的架构级表达

高内聚、低耦合 — 软件模块化设计的永恒目标